

MODULE-2

Informed Search Algorithms

BCSE306

Artificial Intelligence

Dr.Priyanka N

Assistant Professor Senior Grade-1

School of Computer Science and Engineering (SCOPE),

VIT Vellore,

Priyanka.n@vit.ac.in

Informed Search Algorithms

- The algorithms have **information on the goal state**, which helps in more efficient searching.
- This information is obtained by something called a **heuristic Search**.
- Informed search algorithm contains an **array of knowledge** such as **how far we are from the goal**, **path cost**, **how to reach to goal node**, etc.
- This **knowledge** help agents to **explore less to the search space** and find more efficiently the goal node.
- The informed search algorithm is **more useful for large search space**.

Heuristics function

- **Heuristic is a function** which is used in Informed Search, and it finds the most promising path.
- It **takes the current state of the agent** as its input and **produces the estimation of how close agent is from the goal**.
- The heuristic method, however, **might not always give the best solution**, but it **guaranteed to find a good solution in reasonable time**.
- Heuristic function **estimates how close a state is to the goal**.
- It is **represented by $h(n)$** , and it calculates the **cost of an optimal path between the pair of states**.
- The value of the **heuristic function is always positive**.

Admissibility of the heuristic function is given as:

$$h(n) \leq h^*(n)$$

Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost

Pure Heuristic Search

- Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$. It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

Best First Search Algorithm

- Greedy best-first search algorithm always **selects the path which appears best at that moment.**
- It is the **combination of depth-first search and breadth-first search algorithms.**
- It uses the **heuristic function and search.**
- **Best-first search allows us to take the advantages of both algorithms.**
- With the help of best-first search, at each step, we can **choose the most promising node.**
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$f(n) = g(n)$$

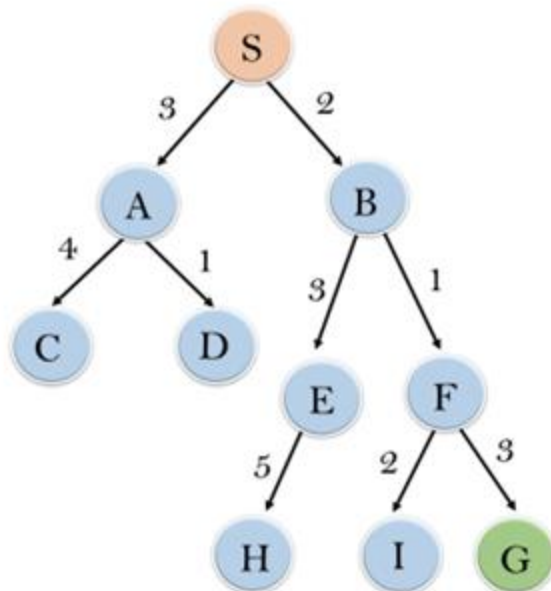
- Where, $h(n)$ = estimated cost from node n to the goal.
- The greedy best first algorithm is implemented by the **priority queue.**

Best first search algorithm:

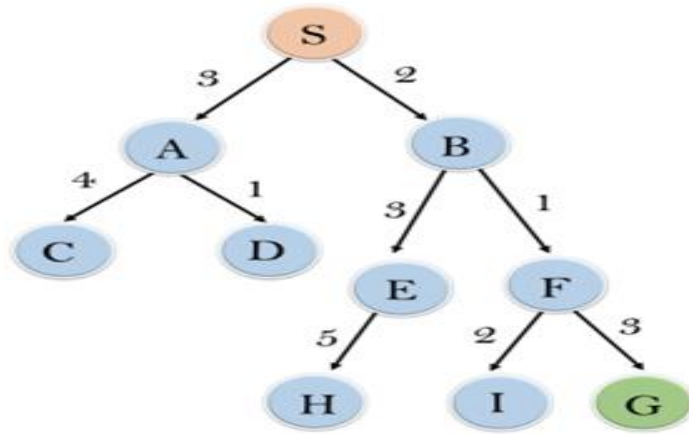
- *Step 1:* Place the starting node into the OPEN list.
- *Step 2:* If the OPEN list is empty, Stop and return failure.
- *Step 3:* Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- *Step 4:* Expand the node n , and generate the successors of node n .
- *Step 5:* Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- *Step 6:* For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- *Step 7:* Return to Step 2.

Example-1

- Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0



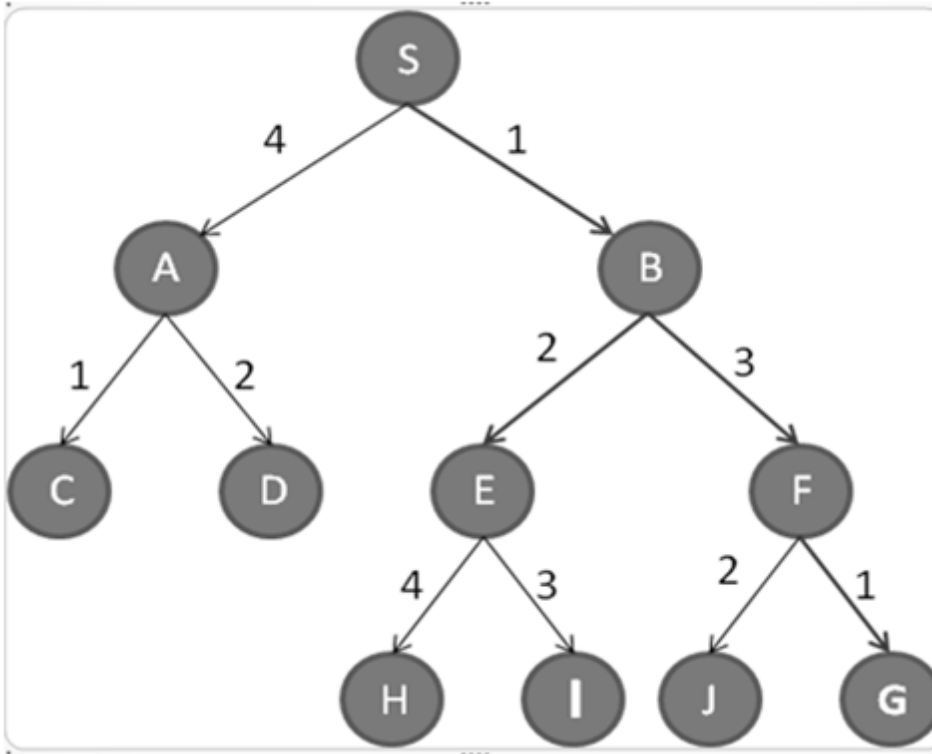
node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Expand the nodes of S and put in the CLOSED list

- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1:** Open [A], Closed [S, B]
- **Iteration 2:** Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]
- **Iteration 3:** Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be:

S----> B----->F-----> G

Example-2



S: Initial state, G: goal.

Table shows the heuristic estimates:

node	$h(n)$	node	$h(n)$	node	$h(n)$
A	11	E	4	I,J	3
B	5	F	2	S	15
C,D	9	H	7	G	0

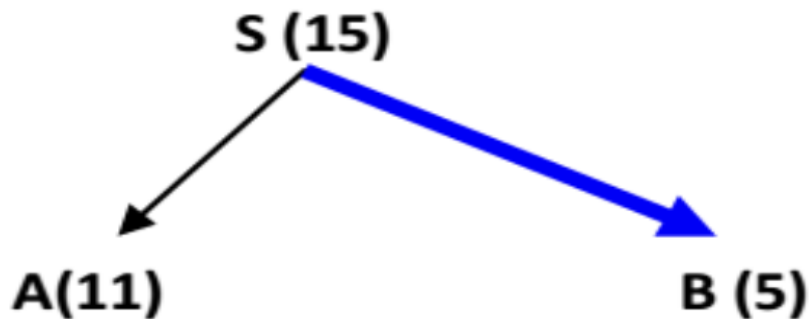
Initialization

S (15)

Expand the nodes of S and add S into **CLOSED**

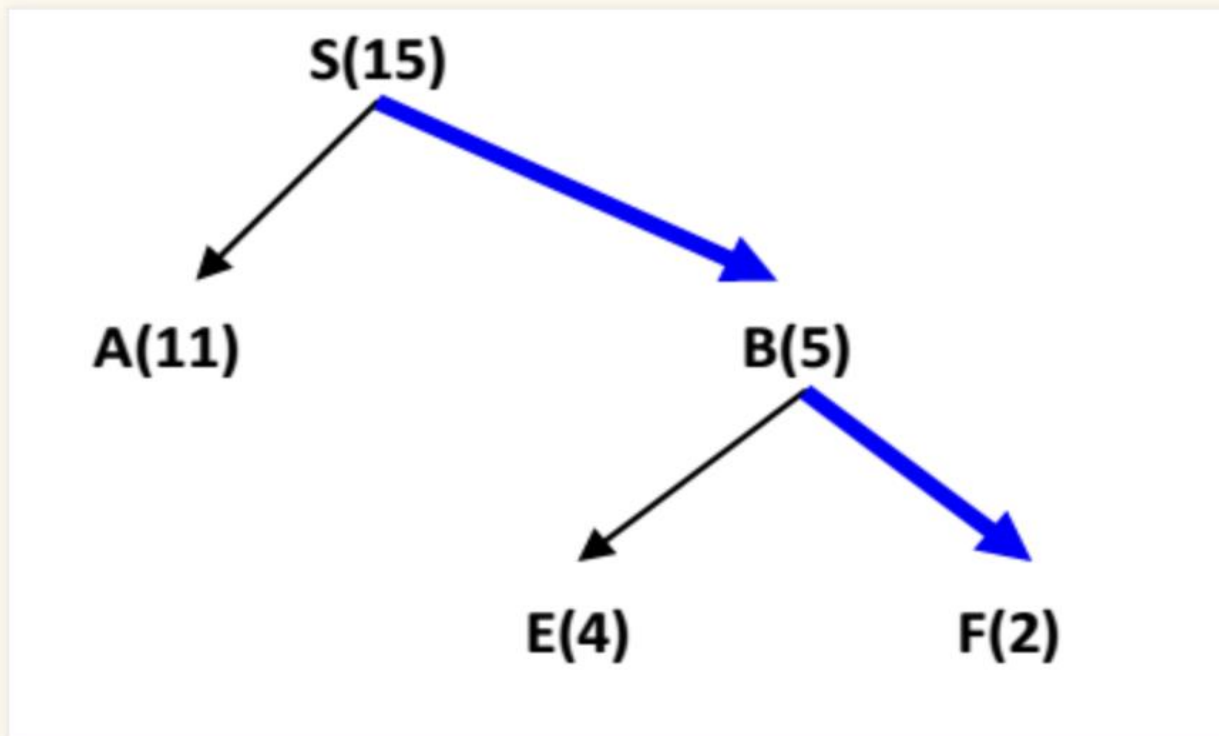
Iteration 1:

- 1) Add the successors of node s into OPEN and compute $f(n)$
- 2) Choose the most promising node ($\min h(n)$)
- 3) Remove the node from OPEN and add it into the CLOSED



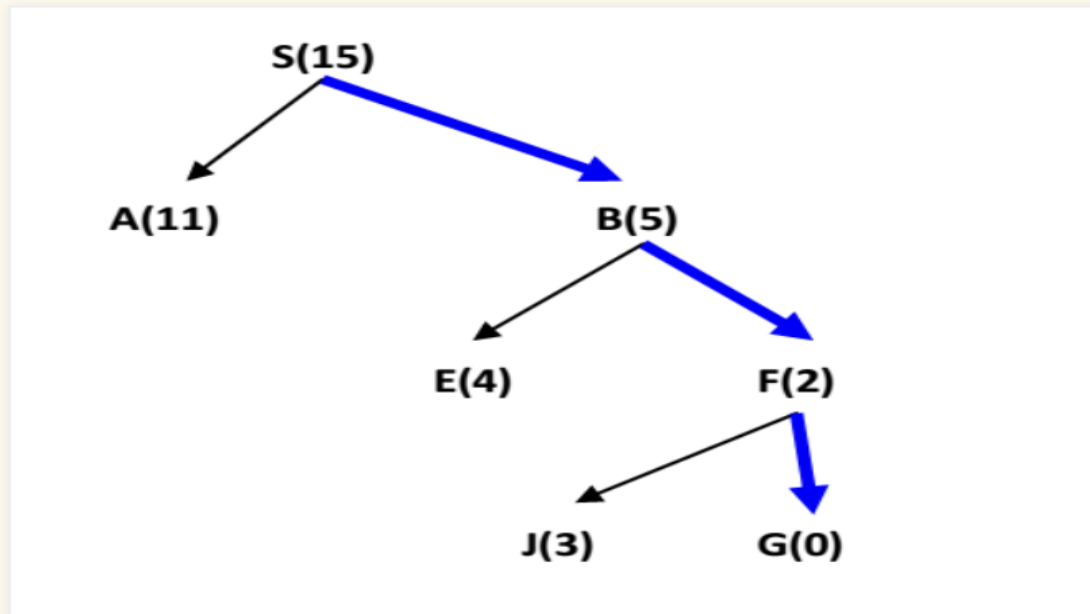
Iteration 2:

- 1) Add the successors of node B into OPEN and compute $f(n)$
- 2) Choose the most promising node ($\min h(n)$)
- 3) Remove the node from OPEN and add it into the CLOSED



Iteration 3:

- 1) Add the successors of node F into OPEN and compute $f(n)$
- 2) Choose the most promising node ($\min h(n)$)
- 3) Remove the node from OPEN and add it into the CLOSED



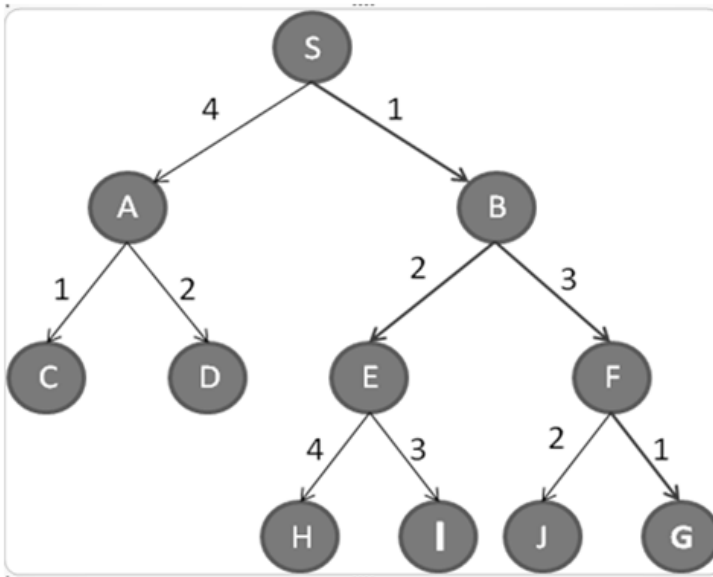
The shortest path from S -> G is S -> B -> F -> G

Total cost:

$$1+3+1=4$$

Iteration	OPEN	CLOSED
Initialization	A,B	S
Iteration 1	A	S,B
Iteration 2	F,E,A	S,B
	E,A	S,B,F
Iteration 3	J,G,E,A	S,B,F
	J,E,A	S,B,F,G

Example-2



S: Initial state, **G:** goal.

Table shows the heuristic estimates:

node	$h(n)$	node	$h(n)$	node	$h(n)$
A	11	E	4	I,J	3
B	5	F	2	S	15
C,D	9	H	7	G	0

Iteration	OPEN	CLOSED
Initialization	A,B	S
Iteration 1	A	S,B
Iteration 2	F,E,A	S,B
	E,A	S,B,F
Iteration 3	J,G,E,A	S,B,F
	J,E,A	S,B,F,G

Advantages

- Best first search can *switch between BFS and DFS* by *gaining the advantages of both* the algorithms.
- This algorithm is *more efficient* than BFS and DFS algorithms.

Disadvantages

- It can behave as an *unguided depth-first search* in the worst-case scenario.
- It can get *stuck in a loop* as DFS.

Performance measure

- **Time Complexity:** The worst-case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst-case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also *incomplete*, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is *not optimal*.